Why FFT produces result in reversed bit order?

FFT notes from sec-bit/mle-pcs

July 2026

Contents

1. What are natural order and reversed bit order?

Let's look at the following table:

indices	bits
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

When we increase the index, the last bit is the most modified bit,

$$\begin{array}{c} \downarrow \\ 101 \end{array}$$

while the first bit is the least modified bit.

$$\begin{array}{c} \downarrow \\ 101 \end{array}$$

The bit in the middle is modified faster than the first bit, but slower than the last bit.

However, in reversed bit order, the first bit is the most modified bit, and the last bit is the least modified bit.

indices	bits
0	000
1	100

indices	bits
2	010
3	110
4	001
5	101
6	011
7	111

This is the most modified bit.

↓
101

This is the least modified bit.

↓
101

Since reversed bit order is reversed by bits compared to natural order, the frequency of the bit flapping is also reversed.

2. Why FFT produces result in reversed bit order?

What if we want the result in natural order?

First, we know what to do in the first level recursion of FFT.

$$f(x) = C_0 + C_1 x + C_2 x^2 + \cdots + C_{n-1} x^{n-1}$$

We divide the polynomial into two parts depending on whether the index of the term is even or odd.

$$f_e(x) = C_0 + C_2 x + C_4 x^2 + \cdots + C_{n-2} x^{n/2-1}$$

$$f_o(x) = C_1 + C_3 x + C_5 x^2 + \cdots + C_{n-1} x^{n/2-1}$$

These two polynomials satisfy the following equation:

$$f(x) = f_e(x^2) + x f_o(x^2)$$

We divide the index of terms simultaneously.

indices	bits
0	000
1	001
2	010
3	011
4	100
5	101

indices	bits
6	110
7	111

into

indices	bits
0	000
2	010
4	001
6	011
—	—
1	100
3	110
5	101
7	111

We can see that we sort the terms by the first bit of their indices.

In the next level recursion, we do the same thing for the second bit. (We only focus on the even terms for simplicity.)

$$f_{ee}(x) = C_0 + C_4 x^4 + \cdots + C_{n-4} x^{n-4}$$

$$f_{eo}(x) = C_2 x^2 + C_6 x^6 + \cdots + C_{n-2} x^{n-2}$$

Again, we divide the index of terms simultaneously.

indices	bits
0	000
4	001
—	—
2	010
6	011
—	—
1	100
5	101
—	—
3	110
7	111

The last level recursion is the same.

$$f_{eee}(x) = C_0 + C_8 x^8 + \cdots + C_{n-8} x^{n-8}$$

$$f_{eeo}(x) = C_4 x^4 + C_{12} x^{12} + \cdots + C_{n-4} x^{n-4}$$

So as indices.

indices	bits
0	000
—	—
4	001
—	—
2	010
—	—
6	011
—	—
1	100
—	—
5	101
—	—
3	110
—	—
7	111

Until now, we apply the required coefficients to the corresponding indices.

As you can see, if we want the result in natural order, we have to apply the coefficients in reversed bit order.

It is because that as we divide the polynomial into two parts, we sort the terms into reversed bit order step by step unconsciously, which is the inner structure of FFT.

So that if we input the coefficients in natural order, then we will get the result in reversed bit order.